Eventify Platform

Phase 2 – Final Implementation Report

Final Submission

SWAPD 351 – Software Architecture and Design

Spring 2026

Team Members

Habiba Magdy
Basmala Salah
Arwa Mohamed
Lana Mohamed

Zewail City of Science and Technology

University of Science and Technology

Document Version 1.0 — March 29, 2026

Contents

Updated Design Document

Revised C4 Model

The C4 model presented in Phase 1 was implemented without structural deviations. The system comprises five microservices (API Gateway, User Service, Event Service, Chat Service, Notification Service) communicating through REST and RabbitMQ, backed by PostgreSQL, MongoDB, and Redis. The Context, Container, and Component diagrams from Phase 1 accurately reflect the deployed system. No changes to the architectural decomposition were necessary during implementation.

Deployment Model

The system is deployed using Docker Compose, orchestrating 14 containers on a single host.

Load Balancing: The API Gateway acts as the single entry point for all client traffic. It distributes incoming requests to the appropriate backend services via HTTP reverse proxy. Nginx serves the React frontend and proxies `/api/*` requests to the API Gateway, providing an additional layer of request distribution.

Failover Mechanisms: Circuit breakers protect all inter-service calls. The Event Service uses `pybreaker` to wrap calls to Redis and RabbitMQ; if either dependency becomes unavailable, the service degrades gracefully (serves uncached responses, skips notifications) rather than failing entirely. Node.js services use `opossum` for the same purpose. RabbitMQ persists messages in durable queues with manual acknowledgment, ensuring that notifications are delivered even if the Notification Service is temporarily down. The User Service implements a startup retry loop (15 attempts, 3-second intervals) to handle database initialization timing.

Scaling Considerations: Each microservice is stateless and can be horizontally scaled via Docker Compose `replicas` or migration to Kubernetes. MongoDB supports horizontal sharding by `event_id` for distributing event data across nodes. PostgreSQL supports sharding via the Citus extension for user data distribution. Redis externalizes session state and caching, allowing any service instance to serve any request without affinity.

Additional ADRs

No additional ADRs were required during implementation. All six decisions from Phase 1 (microservices architecture, polyglot databases, OAuth2/JWT authentication, RabbitMQ message broker, Redis caching, multi-language services) were implemented as specified.

Implemented Web Application

Backend Services

Five microservices were implemented, each following clean architecture with clear separation between controllers, service logic, and data access layers. Each service owns its data store and exposes a RESTful interface.

Service	Language	Responsibilities
API Gateway	Node.js	Request routing via reverse proxy, JWT token validation, rate limiting (100 requests/min per IP), Prometheus metrics exposure
User Service	Node.js	Google OAuth2 via Passport.js, JWT issuance (HS256, 15-min expiry), refresh token management in Redis, user profile CRUD, PostgreSQL storage
Event Service	Python/Flask	Event and RSVP CRUD, MongoDB storage, Redis response caching (5-min TTL) with write-through invalidation, RabbitMQ event publishing, circuit breaker protection via pybreaker
Chat Service	Node.js	Real-time WebSocket messaging via Socket.io, JWT-authenticated connections, per-event chat rooms, MongoDB message persistence, RabbitMQ notification publishing
Notification Service	Python/Flask	RabbitMQ consumer on durable queue, per-user notification storage in Redis, SMTP email delivery (Gmail) triggered on event creation, updates, and RSVPs

Containerization (Docker + Docker Compose)

All services are containerized with individual Dockerfiles and orchestrated via a single `docker-compose.yml` defining 14 containers: 5 application services, 1 frontend, and 8 infrastructure components (PostgreSQL, MongoDB, Redis, RabbitMQ, OpenSearch, OpenSearch Dashboards, Prometheus, Grafana).

CRUD Functionality with API Gateway

The API Gateway proxies all client requests to backend services without consuming request bodies, preserving raw streams for downstream processing. The Event Service implements full CRUD:

- **Create**: `POST /api/events` — validates input, inserts into MongoDB, publishes

`event.created` to RabbitMQ, invalidates Redis cache, triggers email notification to creator.

- **Read**: `GET /api/events` — paginated listing with search and category filtering, served from Redis cache when available. `GET /api/events/{id}` — single event detail with caching.
- **Update**: `PUT /api/events/{id}` — restricted to event creator, publishes `event.updated`, invalidates cache, triggers email.
- **Delete**: `DELETE /api/events/{id}` — restricted to creator, removes associated RSVPs, publishes `event.deleted`, invalidates cache.
- **RSVP**: `POST /api/events/{id}/rsvp` — enforces capacity, increments attendee count atomically, publishes `rsvp.created`, triggers email to user.

Caching (Redis)

Redis serves three functions: (1) response caching for event listings and details with 5-minute TTL and write-through invalidation on mutations; (2) session management storing refresh tokens with 7-day TTL keyed by user ID; (3) notification storage maintaining per-user queues capped at 100 entries.

Message Broker (RabbitMQ)

Asynchronous inter-service communication uses a RabbitMQ topic exchange (`eventify.events`) with durable queues and persistent message delivery. The Event Service and Chat Service publish messages with routing keys (`event.created`, `event.updated`, `event.deleted`, `rsvp.created`, `rsvp.cancelled`, `chat.message`). The Notification Service consumes from a durable queue bound to `event.*`, `rsvp.*`, and `chat.*`, processing messages with manual acknowledgment to guarantee at-least-once delivery.

Centralized Logging (OpenSearch)

OpenSearch 2.11 is deployed as a single-node cluster for centralized log aggregation. OpenSearch Dashboards provides a web interface for log querying and visualization. All services produce structured logs that can be forwarded to OpenSearch for unified search and analysis.

Frontend (React with OAuth2)

A single-page React application built with Bootstrap 5 provides six views: Google OAuth2 login with automatic post-authentication redirect, event browsing with search and category filters, event creation form with field validation, event detail page with role-based actions (RSVP for attendees, delete for creators, chat access for both), real-time WebSocket chat scoped per event, and a live dashboard with 5-second auto-refresh showing total events, total RSVPs, and trending events.

Security Implementation

OAuth2 Authentication (Google)

Authentication uses the OAuth2 Authorization Code flow with Google as the identity provider, implemented via Passport.js. On successful authentication, the User Service creates or retrieves the user in PostgreSQL and issues a signed JWT. The callback redirects the authenticated user to the frontend, which stores the token in `localStorage` for subsequent API calls.

Secure API Implementation

- **Rate Limiting**: 100 requests per minute per IP via `express-rate-limit` at the API Gateway.
- **JWT Tokens**: HS256-signed, 15-minute expiry. The API Gateway validates signatures and injects `x-user-id` headers for downstream authorization.
- **Security Headers**: Helmet.js applies `X-Content-Type-Options`, `X-Frame-Options`, `Strict-Transport-Security`, and `X-XSS-Protection`.
- **Authorization**: Write operations on events are restricted to the creator. RSVP operations require a valid JWT. The Notification Service resolves user emails via internal API calls, never exposing credentials to clients.

Testing and Quality Assurance

Integration Tests

Nine integration tests verify end-to-end behavior across all running services:

Test	Verification
API Gateway health	Returns 200 with correct service identifier
Event Service via Gateway	Proxy forwards requests and returns valid response
List events	Returns paginated response with expected schema
Create event (unauthorized)	Correctly rejects with 401 when no JWT provided
Get nonexistent event	Returns 404 for invalid ObjectId
Event stats	Returns aggregate counts and trending list
Events list contract	Schema: `events` (array), `total` (int), `page` (int)
Stats contract	Schema: `total_events`, `total_rsvps`, `trending_events` (array)
Health contract	Schema: `status` (string), `service` (string)

Result: 9/9 tests passed.

Unit Tests

Unit tests cover core business logic with mocked external dependencies:

- **Event Service**: 10 tests covering health, event listing, creation (with/without auth, missing fields, success), retrieval (not found, invalid ID), RSVP (no auth, not found, duplicate), and statistics.
- **API Gateway**: 4 tests covering health, Prometheus metrics, JWT pass-through, and rate limiting.
- **Notification Service**: 3 tests covering health, notification retrieval, and metrics.

Load and Stress Testing (k6)

A k6 load test simulates progressive concurrency: ramp to 20 users (30s), sustain 50 users (60s), spike to 100 users (30s), ramp down (30s). Thresholds: 95th percentile response time under 500ms, error rate below 5%. The test targets the health, event listing, and statistics endpoints through the API Gateway.

Contract Testing

Three contract tests validate that API response schemas match the structures defined in the OpenAPI specifications, ensuring that independently developed services remain compatible at their interfaces.

Deployment, Monitoring, and Observability

Logging (OpenSearch)

OpenSearch 2.11 provides centralized log storage with full-text search capabilities. OpenSearch Dashboards (port 5601) offers a web interface for log exploration, filtering, and visualization. The cluster operates in single-node mode with security plugins disabled for development. Cluster health was verified as `green` with one active node.

Application and System Monitoring (Prometheus + Grafana)

Prometheus scrapes `/metrics` endpoints from all five application services at 15-second intervals. Each service exposes both default runtime metrics (memory, CPU, event loop latency for Node.js; process metrics for Python) and custom application counters (`http_requests_total` labeled by method, route, and status code; `event_service_requests_total`; `notifications_sent_total` by type). All five scrape targets were confirmed `up` during verification.

Grafana is pre-configured with Prometheus as its default data source via provisioning files. It provides dashboards for request rates, error rates, response latencies, and resource utilization across all services.

Configured Alerts

Alerting is configured through Prometheus rules and RabbitMQ's built-in monitoring:

- **Service availability**: Prometheus alerts when any scrape target transitions to `down`.
- **Error rate**: Alerts when 5xx responses exceed a configurable threshold per service.
- **Queue backlog**: RabbitMQ Management UI monitors queue depth, consumer count, and message rates. Alerts trigger when the notification queue accumulates unprocessed messages beyond a threshold.

Final Report

Summary of System Architecture and Trade-offs

Eventify separates concerns across five microservices by domain: user identity, event management, real-time communication, and notification delivery. Synchronous REST handles user-facing operations requiring immediate feedback. Asynchronous messaging via RabbitMQ decouples notification side-effects from the request path, improving resilience and user-perceived latency.

Decision	Benefit	Cost
Microservices	Independent scaling, fault isolation, polyglot flexibility	Operational complexity, inter-service latency, distributed tracing overhead
Polyglot persistence	Each database optimized for its workload	Two database technologies to operate, no cross-database joins
RabbitMQ	Durable delivery, simple topic routing, sufficient throughput	Not suited for replay or high-volume streaming (Kafka alternative)
JWT authentication	Stateless verification at gateway, no session DB lookup	Revocation requires Redis blacklist or short TTL with refresh
Redis caching	Sub-millisecond reads, reduces database load significantly	Cache invalidation must be carefully synchronized with writes

Challenges Faced and Solutions Implemented

1. **Request body consumption by middleware**: The API Gateway's `express.json()` parsed request bodies before the proxy middleware could forward them, causing empty POST bodies at downstream services. *Solution*: Removed global body parsing; proxy routes receive the unmodified request stream.
2. **Service startup ordering**: The User Service crashed on startup when PostgreSQL was not yet accepting connections. *Solution*: Added a retry loop with 15 attempts at 3-second intervals in the service initialization code.
3. **Notification email delivery**: The notification consumer stored messages in Redis but did not trigger email delivery. *Solution*: Wired the SMTP `send_email` function into the message processing pipeline, with user email resolution via an internal API call to the User Service.
4. **OAuth2 callback redirect**: The Google OAuth2 callback returned raw JSON to the browser instead of redirecting to the React application. *Solution*: Changed the callback handler to issue an HTTP redirect to the frontend URL with the JWT as a query parameter.

Performance Evaluation Results

Operation	Measured Response Time	Target (NFR-02)
Health check	< 5ms	< 200ms
List events (cached)	< 10ms	< 200ms
Create event	< 50ms	< 200ms
RSVP to event	< 30ms	< 200ms
Dashboard statistics	< 20ms	< 200ms
Full E2E test suite (9 tests)	160ms total	–
RabbitMQ message delivery	< 2 seconds	Near real-time
SMTP email delivery	< 5 seconds	Best effort

All measured response times are well within the non-functional requirement targets defined in Phase 1. The RabbitMQ message pipeline delivers notifications to Redis and triggers email within seconds of the originating event. The system sustained all integration and contract tests with zero failures.